AMRITA VISHWA VIDYAPEETHAM

AHEAD Online

Object Oriented Programming Using Java

Objectives

- Runtime Polymorphism/Dynamic binding.

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Dynamic Polymorphism

- Resolves dynamically at the runtime rather than compile-time .
- Also, known as dynamic binding or Dynamic Method Dispatch.
- Achieved by method overriding.
- Multiple methods with the same name and signature in base class and derived class.
- Method invoked is based on object type.

Method Overriding

- Method overriding is the way to achieve dynamic polymorphism in java.
- In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to override the method in the superclass.
- Method to be invoked is determined by the object that is used to invoke at runtime.
- Private methods can not be overloaded.

Method Overriding

- Method overriding is the way to achieve dynamic polymorphism in java.
- When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the subclass.
- The version of the method defined by the superclass will be hidden.

Example

```
class A {
    int i, j;
    A(int a, int b) {
        i = a;
        j = b;
    }
    // Display i and j
    void display() {
        System.out.println("i and j: " + i + " " + j);
    }
}

class B extends A {
    int k;
    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }

    // This overrides display() in A
    void display() {
        System.out.println("k: " + k);
    }
}
```

```
class Override {
    public static void main(String args[]) {
        B subOb = new B(2, 4, 5);
        subOb.display(); // this calls display() in
B
    }
}
```

OUTPUT
k:5

Example

- display() in B will be invoked here and the output will be k=5.
- display() in B overrides the display() of A.
- How to access the super class version?
  - Use super keyword.
  - super.display( ) calls the superclass version of display( )

Dynamic Method Dispatch

- Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time.
- This is how Java implements run-time polymorphism.
- When an overridden method is called through a superclass reference, Java determines which version of that method to execute based upon the type of the object being referred to at the time the call occurs.

Example

```
class A {
    void callme() {
        System.out.println("Inside A's callme method");
    }
}

class B extends A {
    // override callme()
    void callme() {
        System.out.println("Inside B's callme method");
    }
}

class C extends A {
    // override callme()
    void callme() {
        System.out.println("Inside C's callme method");
    }
}
```

```
class Dispatch {
    public static void main(String args[]) {
        A a = new A(); // object of type A
        B b = new B(); // object of type B
        C c = new C(); // object of type C
        A r; // obtain a reference of type A

        r = a; // r refers to an A object
        r.callme(); // calls A's version of callme

        r = b; // r refers to a B object
        r.callme(); // calls B's version of callme

        r = c; // r refers to a C object
        r.callme(); // calls C's version of callme
    }
}
```

Example

- Subclasses B and C override callme( ) declared in A.
- Inside the main( ) method, objects of type A, B, and C and a reference of A are declared.
- Then assigns a reference to each type of object to r and uses that reference to invoke callme( ).
- The output will be

  Inside A's callme method

  Inside B's callme method

  Inside C's callme method

Takeaways

- Method Overriding in Java.

Reference

[1] Naughton P. and Schildt H., “Java2 Complete Reference”, Eighth Edition, Tata McGraw- Hill, 2011.

[2] Ali Bahrami, “Object Oriented Systems Development”, Second Edition, McGraw-Hill, 2008